

TASK PLAN

Library Management System

The library management system is a software to manage the manual functions of a library. The software helps to manage the entire library operations from maintaining book records to issue a book. In addition, it allows streamlined management of fine details of books such as author name, edition, and many other important details. So, it is easier to search for books and find the right materials for students and the librarian.

Electronic management via the software is essential to track information like issue date, due date, who has borrowed any material, etc. The system is developed and designed with an aim to facilitate efficient management of the schools to manage a modern library with accurate data management.

Components of a Library Management System

In order to maintain library management software, you will have the following set of components. These components are efficient to manage library operations accurately.

Admin: the administrators can access the entire functionality of the system via this component. The admin can maintain the records and track them as necessary. Also, the admin can add or remove entries into the system respectively.

Reader: the students who want to access library materials must do registration first. The registration allows for maintaining records accurately. After registering, they can check out and check in the library material.

Book: The admin can add new books or other materials to the system with the essential details. Each book has authno, isbn number, title, edition, category, PublisherID and price.

Publisher: The publisher has PublisherId, Year of publication and name.

Report: It has UserId, Reg_no, Book_no and Issue/Return date. Admin can view the issued materials with their due date. And, if any book is overdue, the system will allow calculating fine for the same.

Relation

- Book(authno, isbn number, title, edition, category, PublisherID, price)
- Reader(UserId, Email, address, phone no, name)
- Publisher(PublisherId, Year of publication, name)
- Report(UserId, Reg_no, Book_no, Issue/Return date)
- Admin (LoginId, password)

TASK 1: Conceptual Design through FTR

CO1, S3

(Tool: Creately/ERD Plus ,ALM:Think pair share)

Using basic database design methodology and ER modeler, design Entity Relationship

Diagram by satisfying the following sub tasks:

1. a Identifying the entities.

1. b Identifying the attributes.

Sample Output

- Publisher(PublisherId, Year of publication, name)
- Admin(LoginId, password, name, staff_id)
- Report(UserId, Reg_no, Book_no, Issue/Return date)
- Book(authno, isbn number, title, edition, category, PublisherID, price)
 - Name is composite attribute of firstname and lastname.
 - Phone no is multi valued attribute.
- Reader(UserId, Email, address, phone no, name)

• c Identification of relationships, cardinality, type of relationship.

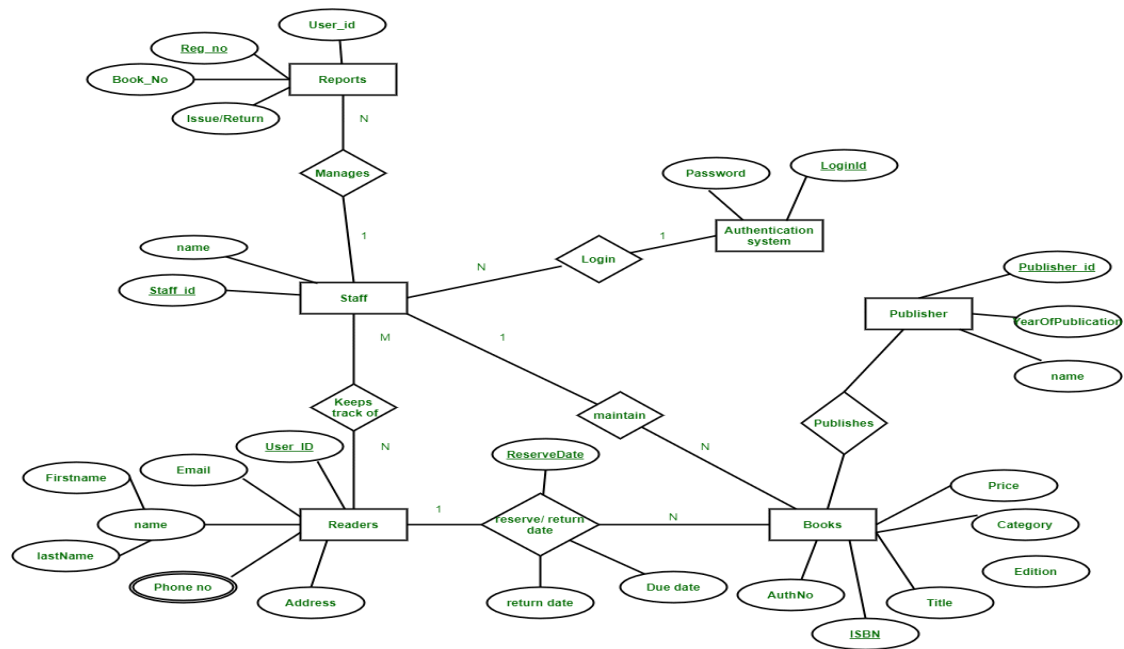
Sample Output

- A reader can reserve N books but one book can be reserved by only one reader. The relationship 1:N.
- A publisher can publish many books but a book is published by only one publisher. The relationship 1:N.
- Admin keeps track of readers. The relationship is M:N.
- Admin maintains multiple reports. The relationship 1:N.
- Admin maintains multiple Books. The relationship 1:N.
- Admin provides login to multiple staffs. The relation is 1:N.
- d Reframing the relations with keys and constraint.

Sample Output

- {isbn number} is the Primary Key for Book Entity.
- {authno, isbn number, title} – Super key of Book entity.
- {PublisherId} is the foreign key of Book entity.
- {authno,title} – Candidate key of Book entity.
- UserId is the Primary Key for Readers entity.
- PublisherID is the Primary Key for Publisher entity
- LoginID as Primary Key for Authentication system entity.
- Reg_no is the Primary Key of reports entity.

1. e Using create, develop ER diagram.



TASK 2: Generating design of other traditional database model CO1, S3

(Tool: Creately, ALM: Mind map)

Creating Hierarchical /Network model of the database by enhancing the sound abstract data by performing following tasks using forms of inheritance:

2. a Identify the specificity of each relationship, find and form surplus relations.
- 2.b Check is-a hierarchy/ has-a hierarchy and performs generalization and/or specialization relationship.
2. c Find the domain of the attribute and perform check constraint to the applicable.
2. d Rename the relations.
2. e Perform SQL Relations using DDL, DCL commands.
 - Create all relations using DDL comments
 - Alter the table Reader by adding email_id as a field.

Sample output

Syntax: Alter table reader add emailed varchar(20);

Output: Table altered

- Retrieve all the books under the category dbms.
- COMMIT the table after inserting values
- Rollback the table.

TASK 3: Using Clauses, Operators and Functions in queries: CO2, S3

(Tool: SQL/ Oracle, ALM: Fish bowl)

Perform the query processing on databases for different retrieval results of queries using DML, DRL operations using aggregate, date, string, indent functions, set clauses and operators.

- Retrieve all the author who wrote in dbms.
- Retrieve total number of books offered in the category program core
- Retrieve all authno and name who published books after 2000
- Retrieve readers name end with letter 'a'
- Retrieve number of readers studied in each department.

Sample Output:

ECE 800
CSE 850
EEE 1000

- Retrieve all the female readers
- Retrieve all the staff who came library yesterday.

TASK 4: Writing Sub Queries and Join Queries:
CO2, S3

(Tool: SQL/ Oracle, ALM: Fish bowl)

Perform the advanced query processing and test its heuristics using designing of optimal correlated and nested sub queries such as finding summary statistics.

- Retrieve isbnnumber ,authnumber and title of the books published under the course DBMS
- Retrieve reader name,id,dept id,department who studied in department CSE .
- Retrieve number of staff borrowed from each category of course
- Retrieve publisher id, year, name of the book published under the course OS.
- Retrieve all the author name who are all published more than 30 books in the year 2022.
- Retrieve all the male readers name and their emailid

Sample input:

<refer reader, emailid)

Sample output:

name	emailid
Rahul	jeni@gmail.com

TASK 5: Design Datalog query and recursive queries(Tool: SQL/ Oracle) CO2, S2

Make use of Datalog query designing and recursive query for student registered for a course.

Find the prerequisite of object-oriented software engineering using recursive query

Sample input:

WITH RECURSIVE

Coursename(pre1, pre2) AS

```
(
(SELECT cname, pre-sub FROM pre_requisites)
UNION
(SELECT a1.pre1, a2.pre2
FROM coursename a1, Ancestor a2
WHERE a1.pre2 = a2.pre1)
)
SELECT pre1
```

FROM Ancestor
WHERE pre2= 'OOSE';

Sample output:
PROBLEM SOLVING USING C

TASK 6: Procedures, Function and Loops:
CO3, S3

(Tool: SQL/ Oracle)

Programming using PL/SQL Procedures, Functions and loops on Number theory and business scenarios like.

- Write PL/SQL procedure using while loop, printing prime numbers in a range given.
- Write PL/SQL function recursion for factorial finding and calculate nth term.
- Write PL/SQL block without procedure/function to print all even multiples of 4,8 and not of 32 below 500.
- Write a non-recursive procedure for palindrome checking.

Sample input:

Enter input: madam

Sample output:
Madam is Palindrome

- Write a simple loop program to print 1 2 3 vertically using PL/SQL loop

TASK 7: Triggers, Views and Exceptions (Tool: SQL/ Oracle)
CO3, S3

Conduct events, views, exceptions on CRUD operations for restricting phenomenon

- Create a simple trigger before insert or update or delete trigger in student table

Sample input:

<refer user schema>

Sample output:

S_id is inserted successfully

- Create a view of all readers name and emailid who are currently in fourth semester.

Sample input:

<refer reader schema>

create trigger book_copies_deducts

after INSERT

on book_issue

for each row

update book_det set copies = copies - 1 where bid =
new.bid;

Sample output:

```
mysql> insert into book_issue values(1, 100, "Java");  
Query OK, 1 row affected (0.09 sec)
```

```
mysql> select * from book_det;  
+-----+-----+-----+  
| bid | btitle      | copies |  
+-----+-----+-----+  
| 1   | Java        | 9      |  
| 2   | C++         | 5      |  
| 3   | MySQL       | 10     |  
| 4   | Oracle DBMS | 5      |  
+-----+-----+-----+  
4 rows in set (0.00 sec)
```

- Raise an exceptional handling whenever a user tries to identify publisherid that does not exists from Publisher Table.

TASK 8: CRUD operations in Document databases (Tool: MongoDB) **CO3, S3**

Perform Mongoose using NPM design on MongoDB designing document database and performing CRUD operations like creating, inserting, querying, finding, removing operations.

Perform the following tasks of library databases.

- Design mongoDB collection for students
- Insert single student detail at a time

Sample input/output:

- Db.student.insertOne({s_id:VTU12345",Name:"Stephen",year:"3",dept_id:"101",contact no,"9876543210" },
- 1 document inserted
- Db.book.find().pretty();
- Insert multiple students at a time
- Insert all at a time.
- Find students who enrolled Java
- Find coursename which is registered by most number of students

- Delete single student record at a time
- Delete multiple student at a time
- Delete all student at a time who are all registered more than
- Update Faculty at a time who is taken single course
- Update multiple faculty at a time who are all handled compiler design
- Update all faculty at a time

TASK 9: CRUD operations in Graph databases (Tool:Neo4j)
S3

CO3,

Perform GraphQL/Neo4j graph space design for recommendation engines. Also perform CRUD operations like creating, inserting, querying, finding, deleting operations on graph spaces.

- Create a graph database for different categories of courses offered by the department and their pre-requisites
- Create a node for prerequisites

Sample input:

Create (pr: pre-requisites

```
{ ccode:123, cname:"DBMS",pre_id:"12345",pre_sub: "DS"
}
)
```

- Insert a course details in each course category
- Delete a course enrolled by student if not satisfied the prerequisites

TASK 10: Normalizing databases using functional dependencies upto BCNF
CO1, S2

(Tool: GU/ Table Normalization Tool, ALM:Learning by doing)

Upon relational tables created in task-2, perform normalization up to BCNF based on given Dependencies as following for the assumed relations specified:

Sample input

- Book(authno, isbn number, title, edition, category, PublisherID, price)
- Reader(UserId, Email, address, phone no, name)
- Publisher(PublisherId, Year of publication, name)
- Admin(LoginId, password, name, staff_id)
- Report(UserId, Reg_no, Book_no, Issue/Return date)

Isbn number authno,title,publisher id

Isbn number edition,category

Isbn number authno

useridname,contactno,

userid name

PublisherId coursename,credits,category

LoginId name, staff_id

LoginId staff_id

3. a Apply the functional dependency, normalize to 1NF
3. b Normalize the relations using FD+ and α^+
3. c Find the minimal cover, canonical cover.
3. d Normalize to 2NF, add/alter constraints if necessary.
3. e Normalize to BCNF, add/alter constraints if necessary.
3. f Normalize to 3NF, add/alter constraints if necessary.
3. g Perform SQL Relational operations using simple DML queries.

Sample output



TASK 11: Menus, Forms and Reports: S3

CO4,

(Tool: SQL/ Oracle 11g ,ALM:Pick the winner)

For an application, creating and debugging Menus, Forms and reports using Oracle Forms and Report Builder,make a report of students with their details.

Sample input:

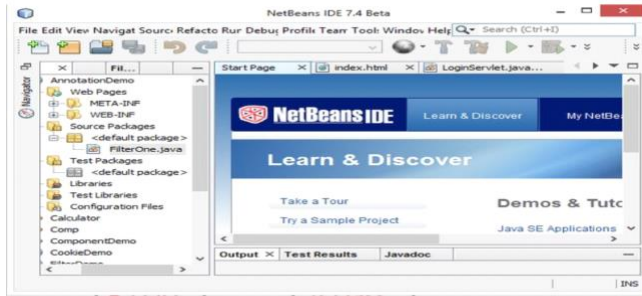
Make a report of students with their details

Sample output:

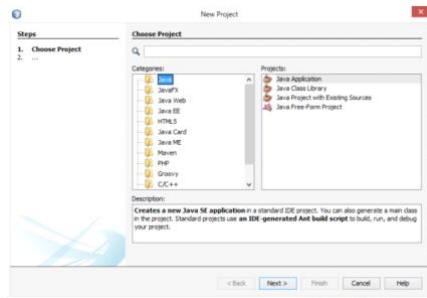
s_id	s_name	year	dept_id	dob	gender	place	phone_no
1234	TEJA	3	301	17/08/2001	M	Chennai	9876543210

xyz@gmail.com

Open the NetBeans IDE.



Choose "Java" -> "Java application" as shown below



**Enter Details**—□×

Book ID(BID)

User ID(UID)

Period(days)

Issued Date(DD-MM-YYYY)

Submit

TASK 12: Micro Project

CO5,

S3

(Tool: Oracle SQL/ SQL Developer/MySQL/MongoDB/NetBeans)

Develop Micro project based on business case scenario on use cases specified in Part-II.

Part-II

****In perusal to the above task, Every student shall select and incorporate for one of the following use cases for Task-12.**

Use case-1: User Module.

This module is further divided into various sub-modules describing the user in a better way:

- **New user register:**
 - To sign up a new user to this system.
- **Student Login:**
 - So as to confirm that only an authenticated user is using the project.
- **Search book:**
 - The user can search book based on book id, book name, or by author name.
- **Issue Book:**
 - To help the user get the required books issued.
- **Return Book:**
 - To return the book before the last date without fine, or after the specified time duration with a late fine.

Use case-2: Admin Module

It is to be operated by the admin with a unique id and password. The admin is the person who decides authentication and authorization for all the different users of the application.

It further can be subdivided as:

- Register user.
- Issue Book.
- Maintain books in a stack, which means record the availability at a regular time interval.

Use case-3: Librarian Module

Includes all the library staff who are required to enter the records in the system and keep an eye on the various activities like the issue of the book, the return of the book, non-availability of books, etc. through the developed system.

TASK 1 - CONCEPTUAL DESIGN THROUGH FTR

Aim

To identify entities attributes and relationship from the usecase given and to draw the ERDiagram for the same.

Tasks and sample output

Using basic database design methodology and ER modeler, design Entity Relationship

Diagram of library management system.

1. a. Identifying the entities.

1. b. Identifying the attributes.

Sample Output

- Publisher(PublisherId, Year of publication, name)
- Admin(LoginId, password, name, staff_id)
- Report(UserId, Reg_no, Book_no, Issue/Return date)
- Book(authno, isbn number, title, edition, category, PublisherID, price)
 - Name is composite attribute of firstname and lastname.
 - Phone no is multi valued attribute.
- Reader(UserId, Email, address, phone no, name)

2. Identification of relationships, cardinality and type of relationship.

Sample Output

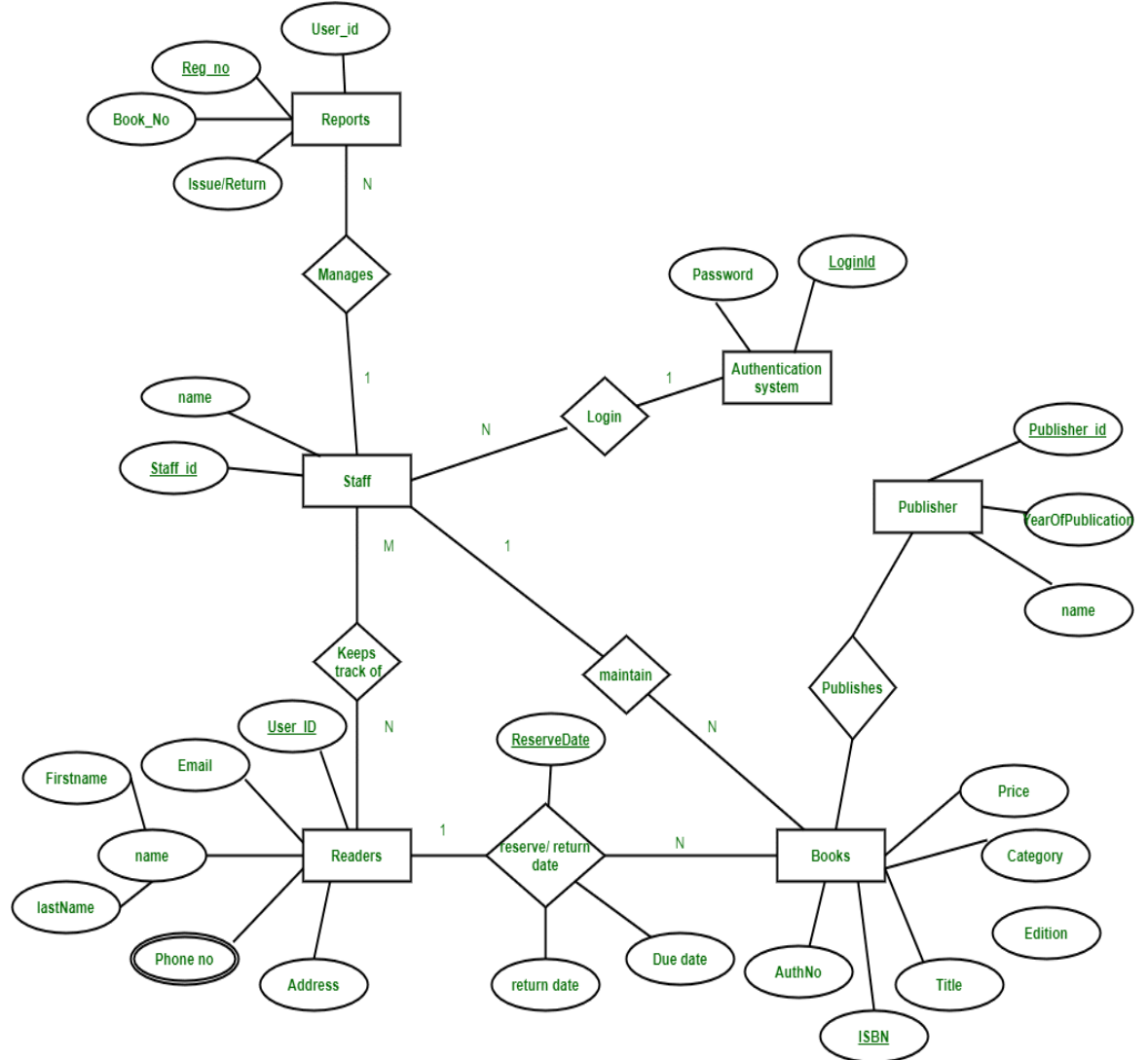
- A reader can reserve N books but one book can be reserved by only one reader. The relationship 1:N.
- A publisher can publish many books but a book is published by only one publisher. The relationship 1:N.
- Admin keeps track of readers. The relationship is M:N.
- Admin maintains multiple reports. The relationship 1:N.
- Admin maintains multiple Books. The relationship 1:N.
- Admin provides login to multiple staffs. The relation is 1:N.

3. Reframing the relations with keys and constraint.

Sample Output

- {isbn number} is the Primary Key for Book Entity.
- {authno, isbn number, title} – Super key of Book entity.
- {PublisherId} is the foreign key of Book entity.
- {authno,title} – Candidate key of Book entity.
- UserId is the Primary Key for Readers entity.
- PublisherID is the Primary Key for Publisher entity
- LoginID as Primary Key for Authentication system entity.
- Reg_no is the Primary Key of reports entity.

- Using createely, develop ER diagram.



RESULT: The task to create an E-R diagram of library management system is executed successfully.

TASK 2 - GENERATING DESIGN OF OTHER TRADITIONAL DATABASE MODEL

Title: Implementation of DDL commands of SQL with suitable examples.

- ☐ Create table
- ☐ Alter table
- ☐ Drop Table

Objective

To understand the different issues involved in the design and implementation of a database system

Theory

Oracle has many tools such as SQL * PLUS, Oracle Forms, Oracle Report Writer, Oracle Graphics etc SQL * PLUS
The SQL * PLUS tool is made up of two distinct parts. These are

- **Interactive SQL:** Interactive SQL is designed for create, access and manipulate data structures like tables and indexes.
- **PL/SQL:** PL/SQL can be used to develop programs for different applications.
 - ☐ **Oracle Forms:** This tool allows you to create a data entry screen along with the suitable menu objects. Thus it is the oracle forms tool that handles data gathering and data validation in a commercial application.
 - ☐ **Report Writer:** Report writer allows programmers to prepare innovative reports using data from the oracle structures like tables, views etc. It is the report writer tool that handles the reporting section of commercial application.
 - ☐ **Oracle Graphics:** Some of the data can be better represented in the form of pictures. The oracle graphics tool allows programmers to prepare graphs using data from oracle structures like tables, views etc.

SQL (Structured Query Language):

Structured Query Language is a database computer language designed for managing data in relational database management systems (RDBMS), and originally based upon Relational Algebra. Its scope includes data query and update, schema creation and modification, and data access control.

SQL was one of the first languages for Edgar F. Codd's relational model and became the most widely used language for relational databases.

- ☐ IBM developed SQL in mid of 1970's.
- ☐ Oracle incorporated in the year 1979.
- ☐ SQL used by IBM/DB2 and DS Database Systems.
- ☐ SQL adopted as standard language for RDBS by ANSI in 1989.

DATA TYPES

- **CHAR (Size):** This data type is used to store character strings values of fixed length. The size in brackets determines the number of characters the cell can hold. The maximum number of character is 255 characters.

- **VARCHAR (Size) / VARCHAR2 (Size):** This data type is used to store variable length alphanumeric data. The maximum character can hold is 2000 character.

- **NUMBER (P, S):** The NUMBER data type is used to store number (fixed or floating point). Number of virtually any magnitude may be stored up to 38 digits of precision. Number as large as 9.99×10^{124} . The precision (p) determines the number of places to the right of the decimal. If scale is omitted then the default is zero. If precision is omitted, values are stored with their original precision up to the maximum of 38 digits.

- **DATE:** This data type is used to represent date and time. The standard format is DD-MM-YY as in 17-SEP-2009. To enter dates other than the standard format, use the appropriate functions. Date time stores date in the 24-Hours format. By default the time in a date field is 12:00:00 am, if no time portion is specified. The default date for a date field is the first day the current month.

- **LONG:** This data type is used to store variable length character strings containing up to 2GB. Long data can be used to store arrays of binary data in ASCII format. LONG values cannot be indexed, and the normal character functions such as SUBSTR cannot be applied.

- **RAW:** The RAW data type is used to store binary data, such as digitized picture or image. Data loaded into columns of these data types are stored without any further

conversion. RAW data type can have a maximum length of 255 bytes. LONG RAW data type can contain up to 2GB.

SQL language is sub-divided into several language elements, including:

- ❑ **Clauses**, which are in some cases optional, constituent components of statements and queries.
- ❑ **Expressions**, which can produce either scalar values or tables consisting of columns and rows of data.
- ❑ **Predicates** which specify conditions that can be evaluated to SQL three-valued logic (3VL) Boolean truth values and which are used to limit the effects of statements and queries, or to change program flow.
- ❑ **Queries** which retrieve data based on specific criteria.
- ❑ **Statements** which may have a persistent effect on schemas and data, or which may control transactions, program flow, connections, sessions, or diagnostics.
- ❑ SQL statements also include the **semicolon** (";") statement terminator. Though not required on every platform, it is defined as a standard part of the SQL grammar.
- ❑ **Insignificant white space** is generally ignored in SQL statements and queries, making it easier to format SQL code for readability.

There are five types of SQL statements. They are:

- DATA DEFINITION LANGUAGE (DDL)
- DATA MANIPULATION LANGUAGE (DML)
- DATA RETRIEVAL LANGUAGE (DRL)
- TRANSACTIONAL CONTROL LANGUAGE (TCL)
- DATA CONTROL LANGUAGE (DCL)

1. DATA DEFINITION LANGUAGE (DDL): The Data Definition Language (DDL) is used to create and destroy databases and database objects. These commands will primarily be used by database administrators during the setup and removal phases of a database project. Let's take a look at the structure and usage of four basic DDL commands:

1. CREATE
2. ALTER
3. DROP
4. RENAME

- **CREATE**

(a) **CREATE TABLE:** This is used to create a new relation (table)

syntax: CREATE TABLE <relation_name/table_name> (field_1 data_type(size),field_2 data_type(size),);

Example:

create table member(membno int, name varchar(20), department varchar(20),gender varchar(10));

Member

membno	name	department	gender
empty			

create table book(isbn int, title varchar(30), authors varchar(10),pagecount int, publisher varchar(10));

Book

isbn	title	authors	pagecount	publisher
empty				

create table borrowed(membno int, isbn int, issuedate date);

Borrowed

membno	isbn	issuedate
empty		

- **ALTER**
- **ALTER TABLE ...ADD...:** This is used to add some extra fields into existing relation.

Syntax: ALTER TABLE relation_name ADD (new field_1 data_type(size), new field_2data_type(size),...);

Example: SQL>Alter table borrowed add name VARCHAR(10)

Book

isbn	title	authors	pagecount	publisher
empty				

- **DROP TABLE:** This is used to delete the structure of a relation. It permanently deletes the recordsin the table.

Syntax: DROP TABLE relation_name;

Example: SQL>DROP TABLE borrowed;

- **RENAME:** It is used to modify the name of the existing database object. Syntax: RENAME TABLEold_relation_name TO new_relation_name;

Example: SQL> ALTER TABLE borrowed RENAME COLUMN name to membername;

Borrowed

membno	isbn	issuedate	membername
empty			

RESULT: The task to create, delete and alter the table are executed successfully.

TASK 3 - USING CLAUSES, OPERATORS AND FUNCTIONS IN QUERIES

Title : Implementation of DML commands using clauses, operators and functions in queries.

- Insert table
- Select table
- Update table
- Delete Table

Objective :

- To understand the different issues involved in the design and implementation of a database system

Theory:

DATA MANIPULATION LANGUAGE (DML): The Data Manipulation Language (DML) is used to retrieve, insert and modify database information. These commands will be used by all database users during the routine operation of the database. Let's take a brief look at the basic DML commands:

1. INSERT 2. UPDATE 3. DELETE

- **INSERT INTO:** This is used to add records into a relation. These are three types of INSERT INTO queries which are as

- **Inserting a single record**

Syntax: INSERT INTO < relation/table name >

(field_1, field_2, ..., field_n) VALUES (data_1, data_2, ..., data_n);

Example: SQL> insert into member values(116,'Shan','CSE','male');

Inserting a single record

Member

membno	name	department	gender
111	jeni	IT	Female
111	Jaz	CSE	Female
113	John	CSE	Female
114	BEn	CSE	Female
115	Ann	CSE	Female
116	Shan	CSE	male

Borrowed

membno	isbn	issuedate
111	11110	2023-01-02
111	11111	2023-02-05
113	11110	2023-02-07

- **UPDATE-SET-WHERE:** This is used to update the content of a record in a relation.

Syntax: SQL>UPDATE relation name SET

Field_name1=data,field_name2=data, WHERE field_name=data;

Example: SQL>UPDATE member SET sname = 'kumar' WHERE sno=1;

- **DELETE-FROM:** This is used to delete all the records of a relation but it will retain the structure of that relation.

- **DELETE-FROM:** This is used to delete all the records of relation.

Syntax: SQL>DELETE FROM relation_name;

Example: SQL>DELETE FROM std;

- **DELETE -FROM-WHERE:** This is used to delete a selected record from a relation.

Syntax: SQL>DELETE FROM relation_name WHERE condition;

Example: SQL>DELETE FROM student WHERE sno = 2;

5. **TRUNCATE:** This command will remove the data permanently. But structure will not be removed.

Difference between Truncate & Delete

By using truncate command data will be removed permanently & will not get back where as by using delete command data will be removed temporally & get back by using roll back command.

By using delete command data will be removed based on the condition where as by using truncate command there is no condition.

Truncate is a DDL command & delete is

a DML command. **Syntax:** TRUNCATE

TABLE <Table name> **Example:**

TRUNCATE TABLE

student;

Sample Queries and Output

- Retrieve member name end with letter 'n' and member no between 111 and 115.

Query:

```
SELECT first_name, last_name, salary FROM employees
WHERE first_name LIKE '%m';
```

Output:

Output

name
John
BEn
Ann

- List the books where pagecount between 700 and 800 – between clause, and operator.

Query:

```
select * from book where pagecount between 700 and 800;
```

Output

isbn	title	authors	pagecount	publisher
11111	Operating Systems	Silberchatz	750	Mcgraw

- Find the records who has minimum number of pagecount – Aggregate

Query:

select min(pagecount) from book;

Output

min(pagecount)
330

- Find the records whose issue date greater than or equal to 07-02-2023

Query:

select * from borrowed where issuedate >='2023-02-07';

Output

membno	isbn	issuedate
113	11110	2023-02-07

- List the membno, but the same membno are listed ones.

Query:

select distinct membno from member;

membno
111
113
114
115
116

- Combine the records of member and book relation – Union

Query:

SELECT membno FROM member UNION SELECT membno FROM

Output

membno
111
113
114
115
116
117
121

borrowed;

- Groupby the member number based on their gender and department.

Query

SELECT membno FROM member GROUP BY gender,department;

Output

membno

111

111

116

117

- Find the authors and their publication details using groupby and orderby clauses.

Query

```
SELECT authors,publisher, COUNT( * ) no FROM book GROUP BY authors, publisher  
order by authors
```

Output

authors	publisher	No
MArk allen	Springer	1
Silberchatz	Mcgraw	1
Silberchatz	Springer	1
William Stallings	Ferrouzan	1

RESULT: The task to implement the DML commands are executed successfully.

TASK 4 - USING FUNCTIONS IN QUERIES AND WRITING SUBQUERIES

- A Subquery or Inner query or a Nested query is a query within another SQL query and embedded within the WHERE clause.
- A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.
- Subqueries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc.

There are a few rules that subqueries must follow –

- Subqueries must be enclosed within parentheses.
- A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.
- An ORDER BY command cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY command can be used to perform the same function as the ORDER BY in a subquery.
- Subqueries that return more than one row can only be used with multiple value operators such as the IN operator.
- The SELECT list cannot include any references to values that evaluate to a BLOB, ARRAY, CLOB, or NCLOB.
- A subquery cannot be immediately enclosed in a set function.
- The BETWEEN operator cannot be used with a subquery. However, the BETWEEN operator can be used within the subquery.

Subqueries with the SELECT Statement

Syntax:

```
SELECT column_name [, column_name ]  
FROM table1 [, table2 ]  
WHERE column_name OPERATOR  
(SELECT column_name [, column_name ]  
FROM table1 [, table2 ]  
[WHERE])
```

- **Write the subqueries to display the isbn number for the records whose page count is less than 500.**

```
SELECT *
FROM book
WHERE isbn IN (SELECT isbn
FROM book
WHERE pagecount > 500) ;
```

Output

isbn	title	authors	pagecount	publisher
1111	OS	Silberchatz	850	springer

- **Select the title and authors whose pagecount is minimum using subquery and functions.**

Query

```
SELECT title,authors FROM book
WHERE pagecount = (SELECT MIN(pagecount) FROM book)
```

Output

title	authors
DBMS	Silberchatz

- **Write a subquery to return the values from multiple tables.**

Query

```
SELECT name, department FROM member WHERE membno == (SELECT isbn FROM
book)
```

Output

name	department
jeni	CSE

Subqueries with the INSERT Statement

```
INSERT INTO table_name [ (column1 [, column2 ]) ]
SELECT [ *|column1 [, column2 ]
FROM table1 [, table2 ]
[ WHERE VALUE OPERATOR ]
```

- **Insert all records in book table into bookduplicate table.**

```
INSERT INTO bookduplicate
SELECT * FROM book
WHERE title IN (SELECT title
FROM book) ;
```

Bookduplicate

isbn	title	authors	pagecount	publisher
111	DBMS	Silberchatz	250	springer
1111	OS	Silberchatz	850	springer

Subqueries with the UPDATE Statement

Syntax

```
UPDATE table
SET column_name = new_value
[ WHERE OPERATOR [ VALUE ]
(SELECT COLUMN_NAME
FROM TABLE_NAME)
[ WHERE) ]
```

- **updates pagecount by 2 times in the book table for all the customers whose pagecount is greater than or equal to 300.**

```
UPDATE book SET pagecount = pagecount * 2
WHERE title IN (SELECT title FROM bookduplicate WHERE
pagecount <= 300 );
```

Book

isbn	title	authors	pagecount	publisher
111	DBMS	Silberchatz	500	springer
1111	OS	Silberchatz	850	springer

Subqueries with the DELETE Statement

```
DELETE FROM TABLE_NAME
[ WHERE OPERATOR [ VALUE ]
(SELECT COLUMN_NAME
FROM TABLE_NAME)
[ WHERE) ]
```

- **Deletes the records from the member table for all the members whose gender is equal to male.**


```
DELETE FROM member WHERE gender IN (SELECT gender
FROM member WHERE gender == 'male' );
```

Output

Member

membno	name	department	gender
111	jeni	CSE	female
112	jaz	IT	female

RESULT: The implementation of SQL commands using functions and subqueries is executed successfully.

TASK 5 - WRITING JOIN QUERIES, EQUIVALENT, AND/OR RECURSIVE QUERIES

Title : Implementation of different types of Joins and recursive queries.

- A SQL JOIN combines records from two tables.
- A JOIN locates related column values in the two tables.
- A query can contain zero, one, or multiple JOIN operations.
- INNER JOIN is the same as JOIN; the keyword INNER is optional.

Aim:

To implement and execute JOIN queries, equivalent queries, and recursive queries using a university database scenario.

Procedure:

The SQL Joins clause is used to combine records from two or more tables in a database. A JOIN is a means for combining fields from two tables by using values common to each. The join is actually performed by the 'where' clause which combines specified rows of tables

- Create the database and tables (Students, Departments, Courses, Enrollments).
- Insert sample data.
- Write SQL queries using different types of **JOINS**.
- Write **equivalent queries** (different approaches to get the same result).

- Implement a **recursive query** (using WITH RECURSIVE).
- Display results and verify correctness.

Step1:

Syntax:

```
SELECT column 1, column 2, column 3...FROM table_name1, table_name2 WHERE
table_name1.column name = table_name2.columnname;
```

Types of Joins :

- Simple Join
- Self Join
- Outer Join

Simple Join:

It is the most common type of join. It retrieves the rows from 2 tables having a common column and is further classified into

Equi-join :

A join, which is based on equalities, is called equi-join.

Example:

```
Select * from item, cust where item.id=cust.id;
```

In the above statement, item-id = cust-id performs the join statement. It retrieves rows from both the tables provided they both have the same id as specified by the where clause. Since the where clause uses the comparison operator (=) to perform a join, it is said to be equijoin. It combines the matched rows of tables. It can be used as follows:

- To insert records in the target table.
- To create tables and insert records in this table.
- To update records in the target table.
- To create views.

Non Equi-join:

It specifies the relationship between columns belonging to different tables by making use of relational operators other than '='.

Example:

```
Select * from item, cust where item.id<cust.id;
```

Table Aliases

Table aliases are used to make multiple table queries shorter and more readable. We give an alias name to the table in the 'from' clause and use it instead of the name throughout the query.

Self join:

Joining of a table to itself is known as self-join. It joins one row in a table to another.

It can compare each row of the table to itself and also with other rows of the same table.

Example:

```
select * from emp x ,emp y where x.salary >= (select avg(salary) from x.emp where
x. deptno
=y.deptno);
```

Outer Join:

It extends the result of a simple join. An outer join returns all the rows returned by simple join as well as those rows from one table that do not match any row from the table. The symbol (+) represents outer join.

Different Types of SQL JOINS

Here are the different types of the JOINS in SQL:

- **(INNER) JOIN:** Returns records that have matching values in both tables

```
SELECT column_name(s) FROM table1
```

```
INNER JOIN table2 ON table1.column_name = table2.column_name;
```

- **LEFT (OUTER) JOIN:** Return all records from the left table, and the matched records from the right table.

```
SELECT column_name(s) FROM table1
```

```
LEFT JOIN table2 ON table1.column_name = table2.column_name;
```

- **RIGHT (OUTER) JOIN:** Return all records from the right table, and the matched records from the left table.

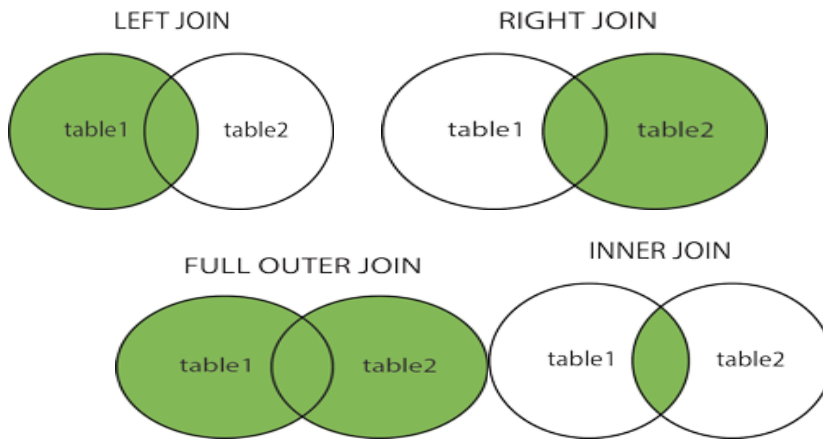
```
SELECT column_name(s) FROM table1
```

```
RIGHT JOIN table2 ON table1.column_name = table2.column_name;
```

- **FULL (OUTER) JOIN:** Return all records when there is a match in either left or right table

```
SELECT column_name(s) FROM table1
```

```
FULL OUTER JOIN table2 ON table1.column_name = table2.column_name;
```



The SQL Joins clause is used to combine records from two or more tables in a database. A JOIN is a means for combining fields from two tables by using values common to each.

Consider the following two tables –

Inner Join

The INNER JOIN keyword selects all rows from both tables as long as the condition is satisfied. This keyword will create the result-set by combining all rows from both the tables where the condition satisfies.

Syntax

SELECT *column_name(s)*

FROM *table1*

INNER JOIN *table2*

ON *table1.column_name = table2.column_name;*

Query

SELECT borrowed.membno, member.NAME **FROM** member

INNER JOIN borrowed

ON member.membno = borrowed.membno;

Output

membno	name
111	jeni

Left Join

The **LEFT JOIN** keyword returns all records from the left table (table1), and the matching records from the right table (table2).

Syntax

SELECT *column_name(s)*

FROM *table1*

LEFT JOIN *table2*

ON *table1.column_name = table2.column_name;*

Query

```
SELECT member.NAME, borrowed.membno FROM member  
LEFT JOIN borrowed ON borrowed.membno = member.membno;
```

SQL RIGHT JOIN Keyword

The **RIGHT JOIN** keyword returns all records from the right table (table2), and the matching records from the left table (table1).

Syntax

```
SELECT column_name(s)  
FROM table1  
RIGHT JOIN table2  
ON table1.column_name = table2.column_name;
```

SQL FULL OUTER JOIN Keyword

The **FULL OUTER JOIN** keyword returns all records when there is a match in left (table1) or right (table2) table records.

Tip: **FULL OUTER JOIN** and **FULL JOIN** are the same.

Syntax :

```
SELECT column_name(s)  
FROM table1  
FULL OUTER JOIN table2  
ON table1.column_name = table2.column_name  
WHERE condition;
```

Recursive Queries

Syntax

```
WITH RECURSIVE [cte_name] (column, ...) AS (  
    [non-recursive_term]  
    UNION ALL  
    [recursive_term])  
SELECT ... FROM [cte_name];  
1 : Write a recursive query to create a Multiplication table by 2  
WITH RECURSIVE x2 (result) AS (  
    SELECT 1  
    UNION ALL  
    SELECT result*2 FROM x2)  
SELECT * FROM x2 LIMIT 10;
```

Output

result

1

2

4

8

16

32

64

128

256

512

(10 rows)

Example 2 - Write a recursive query to create a Fibonacci sequence.

```
WITH RECURSIVE fib(f1, f2) AS (  
    SELECT 0, 1  
    UNION ALL  
    SELECT f2, (f1+f2) FROM fib )  
SELECT f1 FROM fib LIMIT 10;
```

f1

0

1

1

2

3

5

8

13

21

34

(10 rows)

Result:

The implementation of SQL commands using Joins and recursive queries are executed successfully.

Task -6

PL/SQL Procedures, functions, Loops

Aim:

To implement PL/SQL Procedures, Functions and loops on Number theory and business scenarios.

Procedure:

PL/SQL is a combination of SQL along with the procedural features of programming languages. It was developed by Oracle Corporation in the early 90's to enhance the capabilities of SQL.

PL/SQL is one of three key programming languages embedded in the Oracle Database, along with SQL itself and Java.

S.No	Sections & Description
1	Declarations This section starts with the keyword DECLARE . It is an optional section and defines all variables, cursors, subprograms, and other elements to be used in the program.
2	Executable Commands This section is enclosed between the keywords BEGIN and END and it is a mandatory section. It consists of the executable PL/SQL statements of the program. It should have at least one executable line of code, which may be just a NULL command to indicate that nothing should be executed.
3	Exception Handling This section starts with the keyword EXCEPTION . This optional section contains exception(s) that handle errors in the program.

Simple program to print a sentence:

Syntax:

```
DECLARE
    <declarations section>
BEGIN
    <executable command(s)>
EXCEPTION
    <exception handling>
END;
```

Program:

```
DECLARE
    message varchar2(20) := 'booking closed';
```

```
BEGIN
    dbms_output.put_line(message);
END;
```

Static input:

```
SQL> set serveroutput on
SQL> declare
  2   x number(5);
  3   y number(5);
  4   z number(9);
  5   begin
  6   x:=10;
  7   y:=12;
  8   z:=x+y;
  9   dbms_output.put_line('sum is' ||z);
 10   end;
 11   /
sum is22

PL/SQL procedure successfully completed.
```

Dynamic Input:


```

SQL> declare
  2   var1 integer;
  3   var2 integer;
  4   var3 integer;
  5   begin
  6   var1:=&var1;
  7   var2:=&var2;
  8   var3:= var1+var2;
  9   dbms_output.put_line(var3);
 10  end;
 11  /
Enter value for var1: 20
old   6: var1:=&var1;
new   6: var1:=20;
Enter value for var2: 30
old   7: var2:=&var2;
new   7: var2:=30;
50

PL/SQL procedure successfully completed.

```

DECLARE

 hid number(3) := 100;

BEGIN

 IF (hid = 10) THEN
 dbms_output.put_line('Value of hid is 10'
);

 ELSIF (hid = 20) THEN
 dbms_output.put_line('Value of hid is 20'
);

 ELSIF (hid = 30) THEN
 dbms_output.put_line('Value of hid is 30'
);

 ELSE
 dbms_output.put_line('None of the values
is matching');

 END IF;

 dbms_output.put_line('Exact value of hid is:
'|| hid);

END;

```

/
None of the values is matching
Exact value of hid is: 100

PL/SQL procedure successfully completed.
DECLARE
    hid number(1);
    oid number(1);
BEGIN
    << outer_loop >>
    FOR hid IN 1..3 LOOP
        << inner_loop >>
        FOR oid IN 1..3 LOOP
            dbms_output.put_line('hid is: ' || hid || ' and oid is: '
|| oid);
        END loop inner_loop;
    END loop outer_loop;
END;
/
hid is: 1 and oid is: 1
hid is: 1 and oid is: 2
hid is: 1 and oid is: 3
hid is: 2 and oid is: 1
hid is: 2 and oid is: 2
hid is: 2 and oid is: 3
hid is: 3 and oid is: 1
hid is: 3 and oid is: 2
hid is: 3 and oid is: 3

```

PL/SQL procedure successfully completed.

Sample program for only procedure:

```
SQL> create or replace procedure csinformation
  2  (c_id in number, c_name in varchar2)
  3  is
  4  begin
  5  dbms_output.put_line('ID: '||c_id);
  6  dbms_output.put_line('Name: '||c_name);
  7  end;
  8  /
```

Procedure created.

```
SQL> exec csinformation(101,'raam');
```

PL/SQL procedure successfully completed.

```
SQL> set serveroutput on;
```

```
SQL> exec csinformation(101,'raam');
```

ID: 101

Name: raam

PL/SQL procedure successfully completed.

Sample program for only function:

```
SQL>create or replace function csinformation
```

```
(h_id in number,c_name in varchar2)
```

```
Return varchar2
```

```
Is
```

```
Begin
```

```
If c_id>200 then
```

```
Return('no booking available');
```

```
Else
```

```
Return('booking open');
```

```
End if;
```

```
End;
```

```
/
```

Function created.

```
SQL> declare
2  msg varchar2(200);
3  begin
4  msg :=csinformation2(102,'raam');
5  dbms_output.put_line(msg);
6  end;
7  /
vehicle available
```

```
SQL> declare
2  msg varchar2(200);
3  begin
4  msg :=csinformation2(206,'raam');
5  dbms_output.put_line(msg);
6  end;
7  /
No vehicle available
PL/SQL procedure successfully completed.
```

TASK 7

Sample program for only Loops

- To print prime number customer id using loops

```

SQL> declare
2   lo number(3);
3   hi number(3);
4   n number(2);
5   m number(2);
6   c number(20);
7   begin
8   dbms_output.put_line('enter the customer id from to limit:');
9   lo:=&lo;
10  hi:=&hi;
11  for n in lo.. hi
12  loop
13  c:=0;
14  for m in 1..n
15  loop
16  if mod(n,m)=0 then
17  c:=c+1;
18  end if;
19  end loop;
20  if c<=2 then
21  dbms_output.put_line(n||'\n');
22  end if;
23  end loop;
24  end;
25  /
Enter value for lo: 101
old 9: lo:=&lo;
new 9: lo:=101;
Enter value for hi: 120
old 10: hi:=&hi;
new 10: hi:=120;
enter the customer id from to limit:
101\n
103\n
107\n
109\n
113\n
PL/SQL procedure successfully completed.

```

- To check the given customer booking number is Armstrong number.

```

SQL> declare
2   bk number(5);
3   s number:=0;
4   r number;
5   len number;
6   m number;
7   begin
8   bk:=&bk;
9   m:=bk;
10  len:=length(to_char(bk));
11  while bk>0
12  loop
13  r:=mod (bk,10);
14  s:=s+power(r,len);
15  bk:=trunc(bk/10);
16  end loop;
17  if
18  m=s
19  then
20  dbms_output.put_line('given number is armstrong');
21  else
22  dbms_output.put_line('given number is not an armstrong');
23  end if;
24  end;
25  /
Enter value for bk: 234
old 8: bk:=&bk;
new 8: bk:=234;
given number is not an armstrong

```

```

Enter value for bk: 1634
old 8: bk:=&bk;
new 8: bk:=1634;
given number is armstrong
PL/SQL procedure successfully completed.

```

TASK 8: Normalizing databases using functional dependencies upto BCNF

(Tool: GU/ Table Normalization Tool, ALM:Jigsaw) CO3, K3

Upon relational tables created in task-2, perform normalization up to BCNF based on given Dependencies as following for the assumed relations specified below.

Employee Database:

1. Identify employee attributes: Employee_ID, Name, Department, Job_Title, Manager_ID, Hire_Date, Salary.
2. Define relational schema: Employee (Employee_ID, Name, Department, Job_Title, Manager_ID, Hire_Date, Salary).
3. Determine functional dependencies (FDs) between attributes:
 - Employee_ID $\rightarrow$ Name, Department, Job_Title, Manager_ID, Hire_Date, Salary

- Department -> Manager_ID
- Manager_ID -> Name

Step 2: Convert to 1NF

1. Eliminate repeating groups or arrays (none in this example).
2. Create separate tables for each repeating group (none in this example).

Step 3: Convert to 2NF

1. Ensure each non-key attribute depends on the entire primary key.
 2. Move non-key attributes to separate tables if they depend on only part of the primary key.
- Create Department table: Department (Department_ID, Manager_ID, Name).
 - Create Employee table: Employee (Employee_ID, Name, Department_ID, Job_Title, Hire_Date, Salary).

Step 4: Convert to 3NF

1. Ensure there are no transitive dependencies.
 2. Move non-key attributes to separate tables if they depend on another non-key attribute.
- Create Manager table: Manager (Manager_ID, Name).
 - Update Department table: Department (Department_ID, Manager_ID).

Step 5: Convert to BCNF

1. Ensure every determinant is a candidate key.
 2. Check for overlapping candidate keys.
 3. Decompose relations to eliminate redundancy.
- No further decomposition needed.

Using Griffith Tool

1. Input relational schema and functional dependencies.
2. Griffith tool generates a dependency graph.
3. Analyze the graph to identify normalization issues.
4. Apply normalization rules to transform the schema.
5. Verify the resulting schema meets BCNF criteria.

Griffith Tool Steps

1. Create a new project in Griffith.
2. Define the relational schema and FDs.
3. Run the "Dependency Graph" tool.
4. Analyze the graph for normalization issues.
5. Apply transformations using the "Normalize" tool.
6. Verify BCNF compliance using the "BCNF Check" tool.

Normalized Schema

1. Employee (Employee_ID, Name, Department_ID, Job_Title, Hire_Date, Salary).
2. Department (Department_ID, Manager_ID).
3. Manager (Manager_ID, Name).

TASK 9: Backing up and recovery in databases CO4, K3

Perform following backup and recovery scenarios.

- a. Recovering a NOARCHIVELOG Database with Incremental Backups
- b. Restoring the Server Parameter File
- c. Performing Recovery with a Backup Control File

Scenario 1: Recovering a NOARCHIVELOG Database with Incremental Backups

-- Step 1: Backup Database

```
BACKUP DATABASE [database_name] TO DISK = 'backup_file.bak' WITH  
NOFORMAT, NOINIT, NAME = 'Full Database Backup', SKIP, REWIND,  
NOUNLOAD, STATS = 10
```

-- Step 2: Create Incremental Backup

```
BACKUP DATABASE [database_name] TO DISK = 'incremental_backup.bak'  
WITH DIFFERENTIAL, NOFORMAT, NOINIT, NAME = 'Incremental Database  
Backup', SKIP, REWIND, NOUNLOAD, STATS = 10
```

-- Step 3: Simulate Data Loss

-- Intentionally delete or modify data.

-- Step 4: Restore Database
RESTORE DATABASE [database_name] FROM DISK = 'backup_file.bak' WITH
REPLACE

-- Step 5: Apply Incremental Backup
RESTORE DATABASE [database_name] FROM DISK =
'incremental_backup.bak' WITH REPLACE

-- Step 6: Recover Database
RECOVER DATABASE [database_name]

-- Step 7: Open Database
ALTER DATABASE [database_name] SET ONLINE

Scenario 2: Restoring the Server Parameter File (SPFILE)

-- Step 1: Backup SPFILE
BACKUP SERVER PARAMETER FILE TO FILE = 'spfile.bak';

-- Step 2: Simulate SPFILE Loss
-- Delete or modify SPFILE.

-- Step 3: Restore SPFILE
STARTUP MOUNT
RESTORE SERVER PARAMETER FILE FROM FILE = 'spfile.bak';
SHUTDOWN
STARTUP

Scenario 3: Performing Recovery with a Backup Control File

-- Step 1: Backup Control File
BACKUP CONTROLFILE TO FILE = 'controlfile.bak';

-- Step 2: Simulate Control File Loss
-- Delete or modify control file.

-- Step 3: Restore Control File
STARTUP MOUNT
RESTORE CONTROLFILE FROM FILE = 'controlfile.bak';
ALTER CONTROLFILE REUSE;

-- Step 4: Recover Database
RECOVER DATABASE USING BACKUP CONTROLFILE;

-- Step 5: Open Database
ALTER DATABASE OPEN RESETLOGS;

SQL Server Commands:

- BACKUP DATABASE
- RESTORE DATABASE
- RECOVER DATABASE
- ALTER DATABASE
- BACKUP SERVER PARAMETER FILE
- RESTORE SERVER PARAMETER FILE
- BACKUP CONTROLFILE
- RESTORE CONTROLFILE

Result:

TASK 10- CRUD OPERATIONS IN DOCUMENT DATABASES

AIM:

To Perform Mongoose using NPM design on MongoDB designing document database and performing CRUD operations like creating, inserting, querying, finding and removing operations.

STEPS:

Step 1)install Mongo db using following link

<https://www.mongodb.com/try/download/community>

Step 2)install Mongosh using the below link

<https://www.mongodb.com/docs/mongodb-shell/#download-and-install-mongosh>

Step 3)To add the MongoDB Shell binary's location to your PATH environment variable:

Open the Control Panel.

In the System and Security category, click System.

Click Advanced system settings. The System Properties modal displays.

Click Environment Variables.

In the System variables section, select path and click Edit. The Edit environment variable modal displays.

Click New and add the filepath to your mongosh binary.

Click OK to confirm your changes. On each other modal, click OK to confirm your changes.

To confirm that your PATH environment variable is correctly configured to find mongosh, open a command prompt and enter the mongosh --help command.

If your PATH is configured correctly, a list of valid commands displays.

Step 4)Open mongo shell 4.0 from
c:\programfiles\mongoDB\server\bin\mongod.exe

Step 5)Type the CRUD(CREATE READ UPDATE DELETE) COMMANDS
GIVEN IN TEXT FILE.

CRUD OPERATIONS

```
db.createCollection("mylab")
```

```
{ "ok" : 1 }
```

```
>
```

```
db.mylab.insertOne( {item:"canvas",qty:100,tags:["cotton"],size: {h:28,w:35.5,uom:"cm"}} )
```

```
{
```

```
"acknowledged" : true,
```

```
"insertedId" : ObjectId("627d13acc73990c074e6397c")
```

```
}
```

```
> db.mylab.find( {item:"canvas"} )
```

```
{ "_id" : ObjectId("627d13acc73990c074e6397c"), "item" : "canvas", "qty" : 100,
```

```
"tags" : [ "cotton" ], "size" : { "h" : 28, "w" : 35.5, "uom" : "cm" } }
```

```
>
```

```
db.mylab.insertMany([ {item:"journal",qty:25,tags:["blank","red"],size: {h:14,w:21,uom:"cm"} }, {item:"mat",qty:85,tags:["gray"],size: {h:27.9,w:35.5,uom:"cm"} }, {item:"mousepad",qty:25,tags:["gel","blue"],size: {h:19,w:22.85,uom:"cm"} } ])
```

```
{
```

```
"acknowledged" : true,
```

```
"insertedIds" : [
```

```
    ObjectId("627d1598c73990c074e6397d"),
```

```
    ObjectId("627d1598c73990c074e6397e"),
```

```
    ObjectId("627d1598c73990c074e6397f")
```

```

    ]
  }
}
> db.mylab.find({}, {item:1, qty:1})
{ "_id" : ObjectId("627d13acc73990c074e6397c"), "item" : "canvas", "qty" : 100 }
{ "_id" : ObjectId("627d1598c73990c074e6397d"), "item" : "journal", "qty" : 25 }
{ "_id" : ObjectId("627d1598c73990c074e6397e"), "item" : "mat", "qty" : 85 }
{ "_id" : ObjectId("627d1598c73990c074e6397f"), "item" : "mousepad", "qty" : 25 }
}
> db.mylab.find({}, {item:1, qty:1}).pretty()
{
  "_id" : ObjectId("627d13acc73990c074e6397c"),
  "item" : "canvas",
  "qty" : 100
}
{
  "_id" : ObjectId("627d1598c73990c074e6397d"),
  "item" : "journal",
  "qty" : 25
}
{ "_id" : ObjectId("627d1598c73990c074e6397e"), "item" : "mat", "qty" : 85 }
{
  "_id" : ObjectId("627d1598c73990c074e6397f"),
  "item" : "mousepad",
  "qty" : 25
}
> db.mylab.find( {item:"canvas"} ).pretty().sort( {item:-1} )
{
  "_id" : ObjectId("627d13acc73990c074e6397c"),
  "item" : "canvas",
  "qty" : 100,
  "tags" : [
    "cotton"
  ],
  "size" : {
    "h" : 28,
    "w" : 35.5,
    "uom" : "cm"
  }
}
> db.mylab.deleteOne( {item:"journal"} )

```

```

...
...
> db.mylab.find({}, {item:1,qty:1}).pretty()
{
  "_id" : ObjectId("627d13acc73990c074e6397c"),
  "item" : "canvas",
  "qty" : 100
}
{
  "_id" : ObjectId("627d1598c73990c074e6397d"),
  "item" : "journal",
  "qty" : 25
}
{ "_id" : ObjectId("627d1598c73990c074e6397e"), "item" : "mat", "qty" : 85 }
{
  "_id" : ObjectId("627d1598c73990c074e6397f"), "item" : "mousepad", "qty" : 25}

```

Result:

The implementation of CRUD operations like creating, inserting, finding and removing operations using MongoDB is successfully executed.

TASK 11- CRUD OPERATIONS IN GRAPH DATABASES

AIM:

To perform CRUD operations like creating, inserting, querying, finding, deleting operations on graph spaces.

- **Create Node with Properties**

Properties are the key-value pairs using which a node stores data. You can create a node with properties using the CREATE clause. You need to specify these properties separated by commas within the flower braces “{ }”.

Syntax

Following is the syntax to create a node with properties.

```
CREATE (node:label { key1: value, key2: value, . . . . . })
```

- **Returning the Created Node**

To verify the creation of the node, type and execute the following query in the dollar prompt.

```
MATCH (n) RETURN n
```

- **Creating Relationships**

We can create a relationship using the CREATE clause. We will specify relationship within the square braces “[]” depending on the direction of the relationship it is placed between hyphen “ - ” and arrow “ → ” as shown in the following syntax.

Syntax

Following is the syntax to create a relationship using the CREATE clause.

```
CREATE (node1)-[:RelationshipType]->(node2)
```

- **Creating a Relationship Between the Existing Nodes**

You can also create a relationship between the existing nodes using the MATCH clause.

Syntax

Following is the syntax to create a relationship using the MATCH clause.

```
MATCH (a:LabeofNode1), (b:LabeofNode2)
WHERE a.name = "nameofnode1" AND b.name = " nameofnode2"
CREATE (a)-[: Relation]->(b)
RETURN a,b
```

- **Deleting a Particular Node**

To delete a particular node, you need to specify the details of the node in the place of “n” in the above query.

Syntax

Following is the syntax to delete a particular node from Neo4j using the DELETE clause.

```
MATCH (node:label {properties . . . . . })
DETACH DELETE node
```

Create a graph database for student course registration, create student and dept node and insert values of properties.

```
create(n:student{Sid: "VTU14500",
Sname:"John",
```

```
deptname:"CSE" }  
)
```

OUTPUT

Added 1 label, created 1 node, set 3 properties, completed after 232 ms.

```
Create(n:student {Sid: "VTU14501",  
Sname:"Dharsana",  
deptname:"EEE"})
```

OUTPUT

Added 1 label, created 1 node, set 3 properties, completed after 16 ms.

```
Create(w:student { Sid: "VTU14502",  
Sname:"vijay",  
deptname:"CSE"  
})
```

OUTPUT

Added 1 label, created 1 node, set 3 properties, completed after 12 ms.

```
Create(n:dept{deptname:"cse",deptid:"d001"})
```

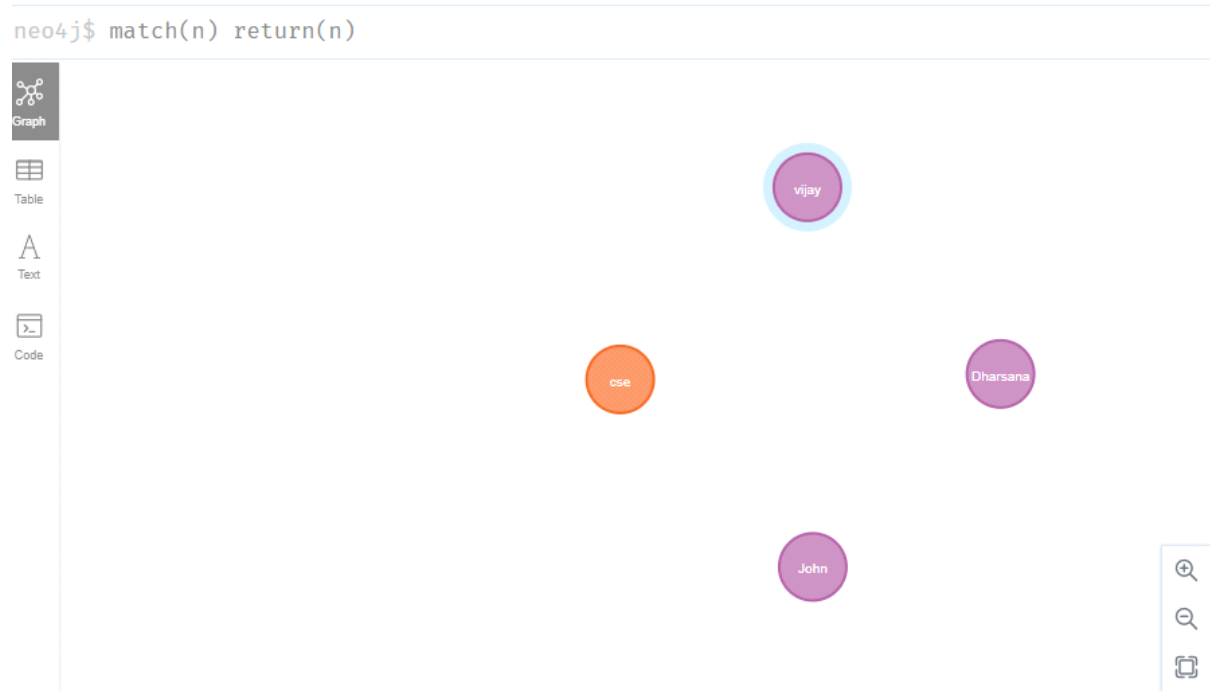
OUTPUT:

Added 1 label, created 1 node, set 2 properties, completed after 72 ms.

Select all the nodes in your database using match command

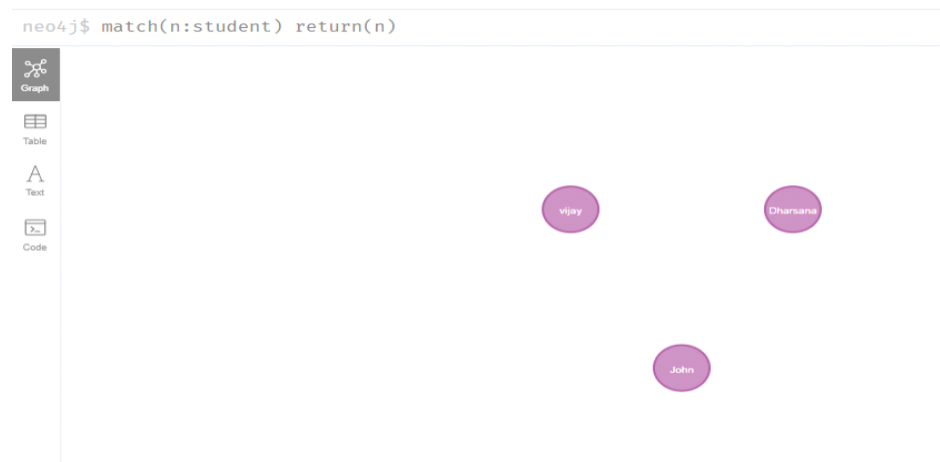
- **match(n) return(n)**

OUTPUT



- **`match(n:student) return(n)`**

OUTPUT:

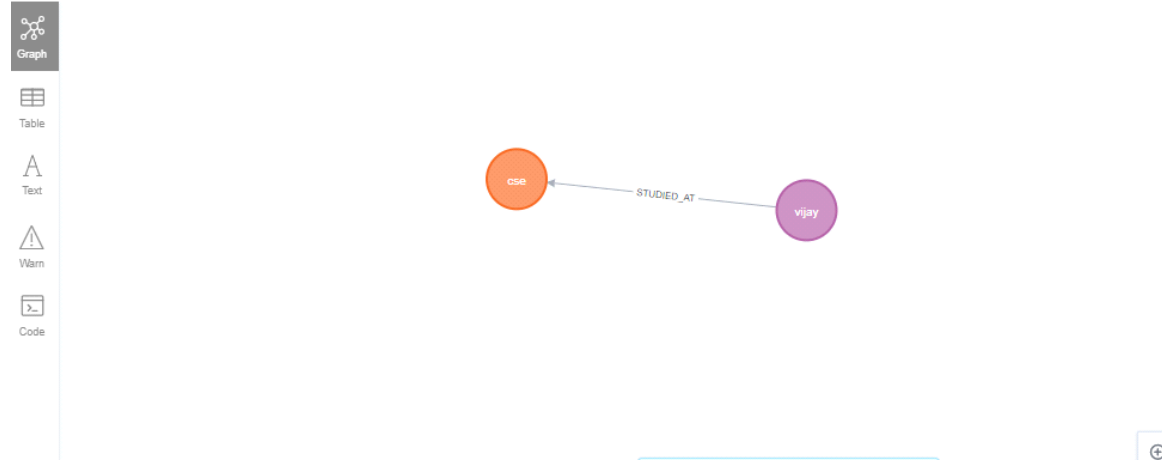


- Create relationship between student and cse .**

```
MATCH(s:student),(d:dept) WHERE s.Sname ='vijay' AND d.deptname='cse'  
CREATE(s)-[st:STUDIED_AT]->(d)  
return s,d
```

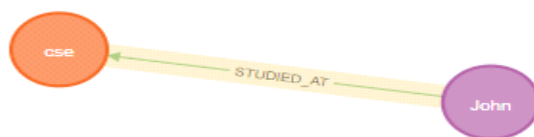

OUTPUT:

```
1 MATCH(s:student),(d:dept) WHERE s.Sname='vijay' AND d.deptname='cse'
2 CREATE(s)-[st:STUDIED_AT]-(d)
3 return s,d
4
5
6
7
8
```



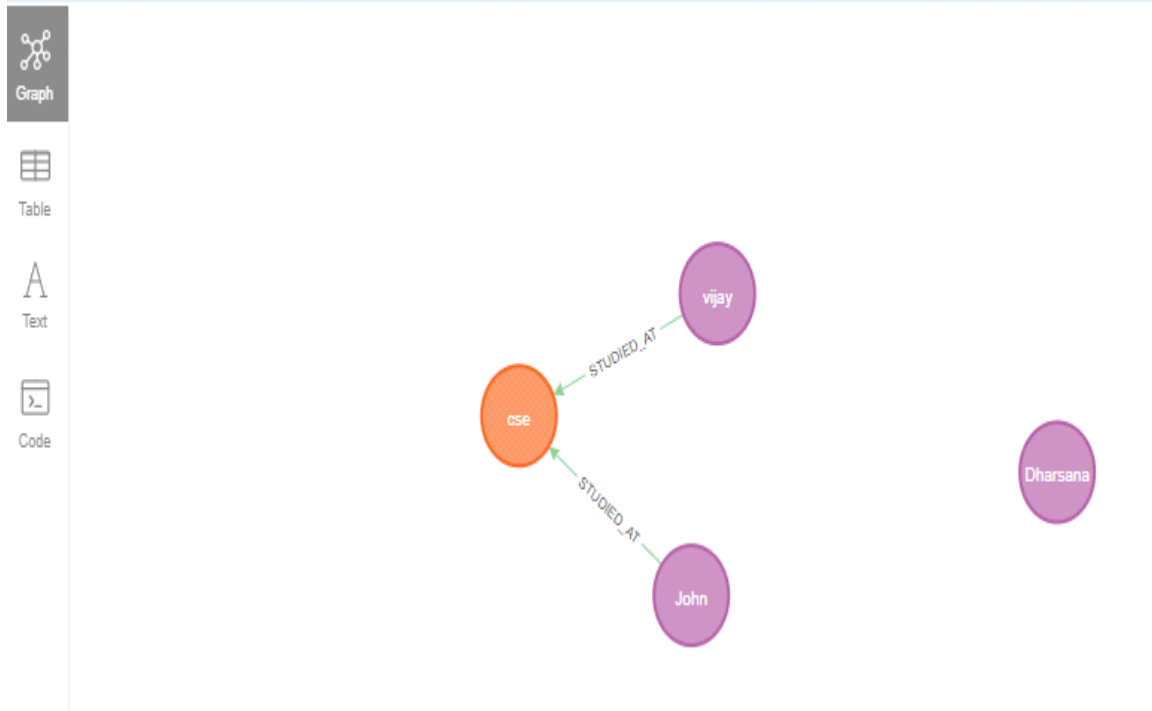
```
MATCH(s:student),(d:dept) WHERE s.Sname='John' AND d.deptname='cse'
CREATE(s)-[st:STUDIED_AT]-(d)
return s,d
```

OUTPUT:



match(n) return(n)

```
neo4j$ match(n) return(n)
```



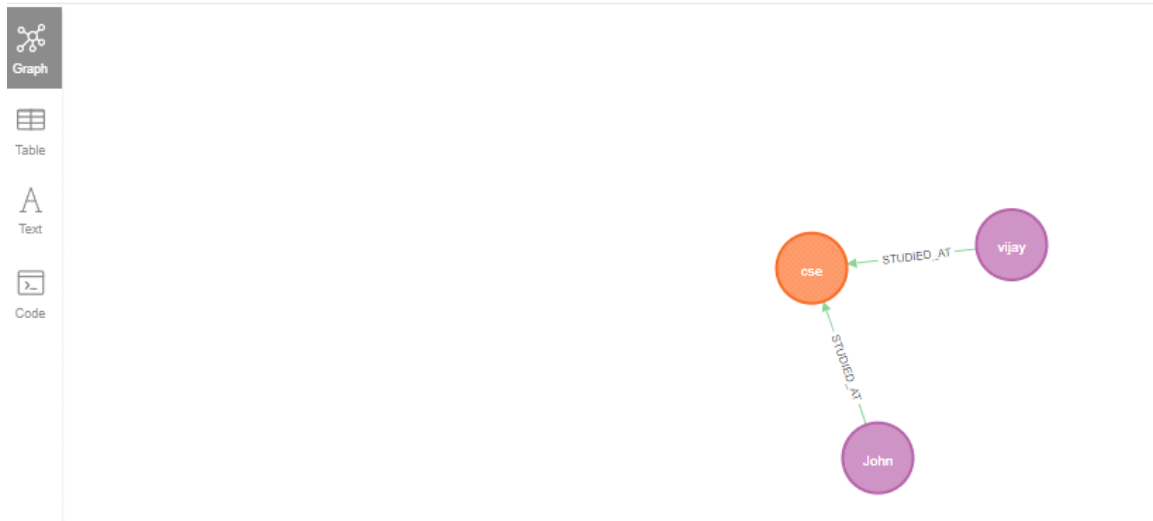
a. Delete a node from student

```
match(n:student{Sname:'Dharsana'}) DELETE(n)
```

OUTPUT:

Deleted 1 node, completed after 10834 ms.

```
neo4j$ match(n)return(n)
```



Result

The implementation of CRUD operations like creating, inserting, finding and removing operations using GraphDB is successfully executed.

TASK 12: MINI PROJECT

Loan Database Management System

AIM: To create a loan database management system.

Entities and Attributes of Loan Automation System

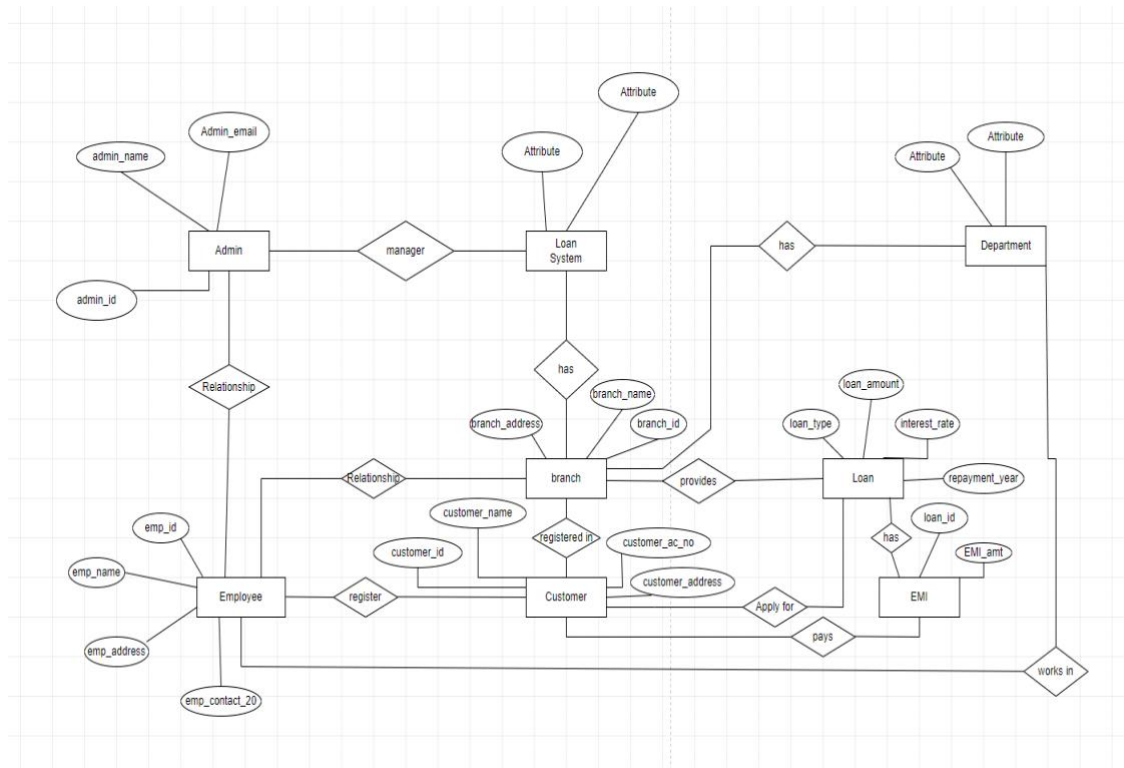
- **Admin:** Admin_id, Admin_name, Admin_email
- **Employee:** Emp_id, Emp_name, Emp_address, Emp_contact_no
- **Customer:** Costomer_id, Costomer_name, Customer_address, Customer_ac_no
- **Branch:** Branch_id, Branch_name, Branch_address
- **Department:** Depart_id, Depart_name
- **Loan:** Loan_type, Loan_amount, Interest_rate
- **Loan System:** No_of_branch, Name_of_branch
- **EMI:** loan_id, EMI_amt

Overview of Loan Automation System Database Working

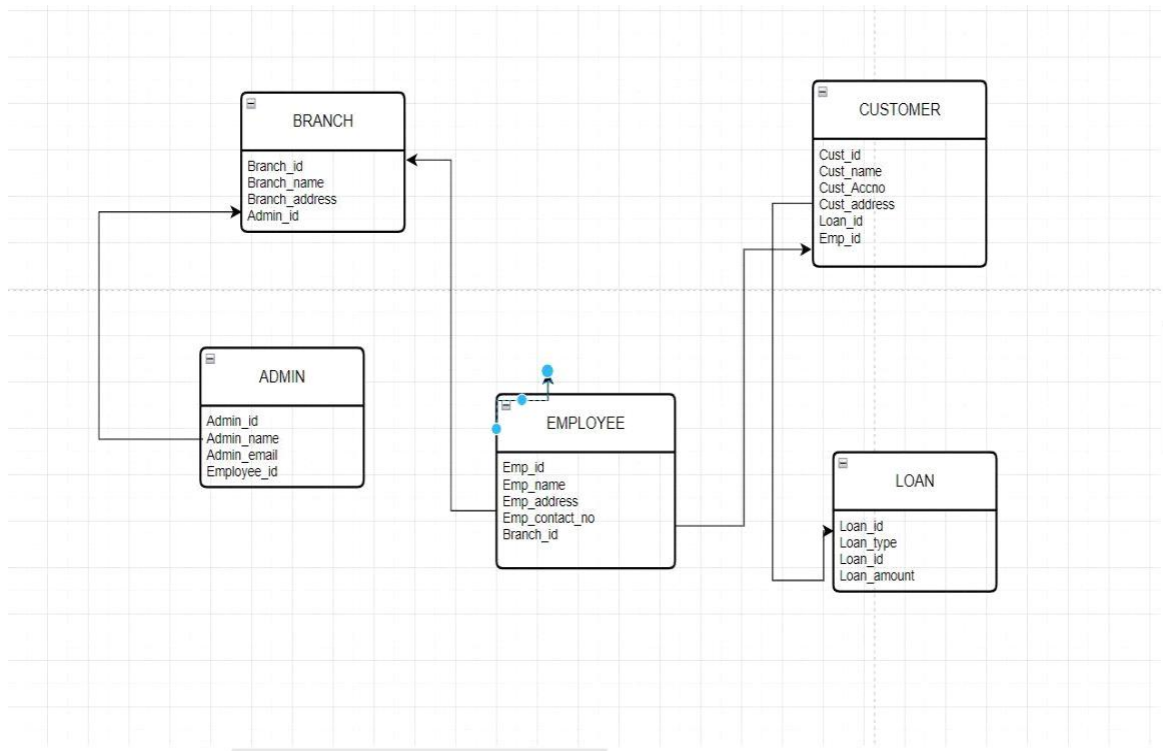
Loan automation system helps to automat loan approval and EMI repayment process. This application is commonly used by loan providing organizations for easy record handling. Some of the main features of this application are as follows:

- Loan automation system works on client server approach
- This application is specifically for loan providing organizations. This is not a general application
- Admin manages the entire loan system. He manages all master records
- Loan system has multiple branches. Each branch has multiple departments
- Admin appointments employees in different branches
- These employee register customer who comes for loan in the branch
- This application also maintain record of loan repayment
- Each loan has a unique loan account number
- Whenever customer pays his loan EMI, it is recorded in customer loan account
- Admin can view all reports such as customer list, loan outstanding report, loan repayment report etc
- Customer can view all paid EMI details

ER MODEL



RELATIONAL TABLE



COMMANDS USED TO CREATE RELATIONAL TABLE

```

SQL*Plus: Release 11.2.0.2.0 Production on Thu Mar 30 20:16:55 2023

Copyright (c) 1982, 2014, Oracle. All rights reserved.

SQL> connect system
Enter password:
Connected.
SQL> create table cust(cust_id int,cust_accno int,cust_name varchar(10),loan_id int,loan_type varchar(10),loan_amount int);

Table created.

SQL> desc cust;

```

Name	Null?	Type
CUST_ID		NUMBER(38)
CUST_ACCNO		NUMBER(38)
CUST_NAME		VARCHAR2(10)
LOAN_ID		NUMBER(38)
LOAN_TYPE		VARCHAR2(10)
LOAN_AMOUNT		NUMBER(38)

```

SQL> insert into cust values(&cust_id,&cust_accno,&cust_name,&loan_id,&loan_type,&loan_amount);
Enter value for cust_id: 1
Enter value for cust_accno: 889977
Enter value for cust_name: anil kumar
Enter value for loan_id: 100
Enter value for loan_type: Education
Enter value for loan_amount: 400000
old 1: insert into cust values(&cust_id,&cust_accno,&cust_name,&loan_id,&loan_type,&loan_amount)
new 1: insert into cust values(1,889977,'anil kumar',100,'Education',400000)

1 row created.

SQL> /
Enter value for cust_id: 2
Enter value for cust_accno: 889978
Enter value for cust_name: dharma
Enter value for loan_id: 101
Enter value for loan_type: Home

SQL> /
Enter value for cust_id: 9
Enter value for cust_accno: 889985
Enter value for cust_name: Chandu
Enter value for loan_id: 109
Enter value for loan_type: Agriculture
Enter value for loan_amount: 500000
old 1: insert into cust values(&cust_id,&cust_accno,&cust_name,&loan_id,&loan_type,&loan_amount)
new 1: insert into cust values(9,889985,'Chandu',109,'Agriculture',500000)insert into cust values(9,889985,'Chandu',109,'A
*

ERROR at line 1:
ORA-12899: value too large for column "SYSTEM"."CUST"."LOAN_TYPE" (actual: 11,
maximum: 10)

SQL> select *from cust;

```

CUST_ID	CUST_ACCNO	CUST_NAME	LOAN_ID	LOAN_TYPE	LOAN_AMOUNT
1	889977	anil kumar	100	Education	400000
2	889978	dharma	101	Home	5000000
3	889979	lohit	102	Marraiage	100000000
4	889980	Srinivas	104	Medical	100000
5	889981	Mastan	103	Bussieness	200000
6	889982	Teja	105	Car	300000
7	889983	Gangupamu	108	Phone	100000
8	889984	Rupesh	106	Education	240000

```

8 rows selected.

```

```

Enter value for loan_amount: 5000000
old 1: insert into cust values(&cust_id,&cust_accno,&cust_name,&loan_id,&loan_type,&loan_amount)
new 1: insert into cust values(2,889978,'dharma',101,'Home',5000000)

1 row created.

SQL> /
Enter value for cust_id: 3
Enter value for cust_accno: 889979
Enter value for cust_name: lohith
Enter value for loan_id: 102
Enter value for loan_type: Marraiage
Enter value for loan_amount: 100000000
old 1: insert into cust values(&cust_id,&cust_accno,&cust_name,&loan_id,&loan_type,&loan_amount)
new 1: insert into cust values(3,889979,'lohith',102,'Marraiage',100000000)
1 row created.

SQL> /
Enter value for cust_id: 4
Enter value for cust_accno: 889980
Enter value for cust_name: Srinivas
Enter value for loan_id: 104
Enter value for loan_type: Medical
Enter value for loan_amount: 1000000
old 1: insert into cust values(&cust_id,&cust_accno,&cust_name,&loan_id,&loan_type,&loan_amount)
new 1: insert into cust values(4,889980,'Srinivas',104,'Medical',1000000)

1 row created.

SQL> /
Enter value for cust_id: 5
Enter value for cust_accno: 889981
Enter value for cust_name: Mastan
Enter value for loan_id: 103
Enter value for loan_type: Bussieness
Enter value for loan_amount: 2000000
old 1: insert into cust values(&cust_id,&cust_accno,&cust_name,&loan_id,&loan_type,&loan_amount)
new 1: insert into cust values(5,889981,'Mastan',103,'Bussieness',2000000)

1 row created.

SQL> /
Enter value for cust_id: 6
Enter value for cust_accno: 889982
Enter value for cust_name: Teja
Enter value for loan_id: 105
Enter value for loan_type: Car
Enter value for loan_amount: 3000000
old 1: insert into cust values(&cust_id,&cust_accno,&cust_name,&loan_id,&loan_type,&loan_amount)
new 1: insert into cust values(6,889982,'Teja',105,'Car',3000000)

1 row created.

SQL> /
Enter value for cust_id: 7
Enter value for cust_accno: 889983
Enter value for cust_name: Gangupamu
Enter value for loan_id: 108
Enter value for loan_type: Phone
Enter value for loan_amount: 1000000
old 1: insert into cust values(&cust_id,&cust_accno,&cust_name,&loan_id,&loan_type,&loan_amount)
new 1: insert into cust values(7,889983,'Gangupamu',108,'Phone ',1000000)

1 row created.

SQL> 8
SP2-0226: Invalid line number
SQL> /
Enter value for cust_id: 8
Enter value for cust_accno: 889984
Enter value for cust_name: Rupesh
Enter value for loan_id: 106
Enter value for loan_type: Education
Enter value for loan_amount: 2400000
old 1: insert into cust values(&cust_id,&cust_accno,&cust_name,&loan_id,&loan_type,&loan_amount)
new 1: insert into cust values(8,889984,'Rupesh',106,'Education',2400000)

1 row created.

```

NORMALISATION

Attributes in Table

❗ Separate attributes using a comma (,)

cust_id,cust_name,cust_Accno,loan_id,loan_type,loan_amount

Functional Dependencies

Enter the functional dependencies that are present in your table.

cust_id ×	→	loan_id ×	Delete
cust_name ×	→	loan_type ×	Delete
cust_id ×	→	loan_amount ×	Delete
cust_Accno ×	→	loan_amount ×	Delete

Normalize to 2NF

Attributes

cust_id loan_id loan_amount

Functional Dependencies

cust_id → loan_id loan_amount

Attributes

cust_name loan_type

Functional Dependencies

cust_name → loan_type

Attributes

cust_id cust_name cust_Accno

Functional Dependencies

Normalize to 3NF

Attributes

cust_id loan_amount loan_id

Functional Dependencies

cust_id $\rightarrow$ loan_amount loan_id

Attributes

cust_name loan_type

Functional Dependencies

cust_name $\rightarrow$ loan_type

Attributes

cust_Accno loan_amount

Functional Dependencies

cust_Accno $\rightarrow$ loan_amount

Attributes

cust_id cust_name cust_Accno

Functional Dependencies

Show Steps



Step 1. Find merged minimal cover of FDs, which contains:
 $\text{cust_id} \rightarrow \text{loan_id, loan_amount}$
 $\text{cust_name} \rightarrow \text{loan_type}$
 $\text{cust_Accno} \rightarrow \text{loan_amount}$

Initially $\text{rel}[1]$ contains the original table, with the FDs above

Round1: Checking whether table $\text{rel}[1]$ is in BCNF

The FD $[\text{cust_id} \rightarrow \text{loan_id, loan_amount}]$ violates BCNF as the LHS is not superkey. Table is split into the two below:

$\text{rel}[2] = (\text{cust_id, loan_id, loan_amount})$
With FDs:

$\text{rel}[3] = (\text{cust_id, cust_name, cust_Accno, loan_type})$
With FDs:

Round2: Checking whether table $\text{rel}[2]$ is in BCNF

*** Table $\text{rel}[2]$ is in BCNF already, send it to output ***

Round3: Checking whether table $\text{rel}[3]$ is in BCNF

The FD $[\text{cust_name} \rightarrow \text{loan_type}]$ violates BCNF as the LHS is not superkey. Table is split into the two below:

$\text{rel}[4] = (\text{cust_name, loan_type})$
With FDs:

$\text{rel}[5] = (\text{cust_id, cust_name, cust_Accno})$
With FDs:

Round4: Checking whether table $\text{rel}[4]$ is in BCNF

*** Table $\text{rel}[4]$ is in BCNF already, send it to output ***

Round5: Checking whether table $\text{rel}[5]$ is in BCNF

*** Table $\text{rel}[5]$ is in BCNF already, send it to output ***

Show Steps



First, find the minimal cover of the FDs, which includes the FDs

$\text{cust_id} \rightarrow \text{loan_id}$
 $\text{cust_name} \rightarrow \text{loan_type}$
 $\text{cust_id} \rightarrow \text{loan_amount}$
 $\text{cust_Accno} \rightarrow \text{loan_amount}$

Initially $\text{rel}[1]$ is the original table:

Round1: checking table $\text{rel}[1]$

The table is not in 2NF.

The FD $[\text{cust_id} \rightarrow \text{loan_id}]$ is a partial dependency (i.e., LHS is a proper subset of some CK), the table is split into:

$\text{rel}[2] = (\text{cust_id}, \text{loan_id}, \text{loan_amount})$, with FDs:
 $\text{cust_id} \rightarrow \text{loan_id}, \text{loan_amount}$

$\text{rel}[3] = (\text{cust_id}, \text{cust_name}, \text{cust_Accno}, \text{loan_type})$, with FDs:
 $\text{cust_name} \rightarrow \text{loan_type}$

Round2: checking table $\text{rel}[2]$

***** The table is in 2NF already, send it to output *****

Round3: checking table $\text{rel}[3]$

The table is not in 2NF.

The FD $[\text{cust_name} \rightarrow \text{loan_type}]$ is a partial dependency (i.e., LHS is a proper subset of some CK), the table is split into:

$\text{rel}[4] = (\text{cust_name}, \text{loan_type})$, with FDs:
 $\text{cust_name} \rightarrow \text{loan_type}$

$\text{rel}[5] = (\text{cust_id}, \text{cust_name}, \text{cust_Accno})$, with FDs:

Round4: checking table $\text{rel}[4]$

***** The table is in 2NF already, send it to output *****

Round5: checking table $\text{rel}[5]$

***** The table is in 2NF already, send it to output *****

Normalize to BCNF

Attributes

cust_id loan_id loan_amount

Functional Dependencies

$\text{cust_id} \rightarrow \text{loan_id} \text{ loan_amount}$

Attributes

cust_name loan_type

Functional Dependencies

$\text{cust_name} \rightarrow \text{loan_type}$

Attributes

cust_id cust_name cust_Accno

Functional Dependencies

RESULT: The task to create a loan database management system is executed successfully.

Viva Questions

- What is DBMS?

DBMS is a collection of programs that facilitates users to create and maintain a database.

- What is a database?

Database is a logical, consistent and organized collection of data that it can easily be accessed, managed and updated.

- What is a database system?

The collection of database and DBMS software together is known as database system.

- What are the advantages of DBMS?

- Redundancy control
- Restriction for unauthorized access
- Provides multiple user interfaces
- Provides backup and recovery
- Enforces integrity constraints

- What is checkpoint in DBMS?

A Checkpoint is like a snapshot of the DBMS state.

- When does checkpoint occur in DBMS?

By taking checkpoints, the DBMS can reduce the amount of work to be done during restart in the event of subsequent crashes.

- What do you mean by transparent DBMS?

The transparent DBMS is a type of DBMS which keeps its physical structure hidden from users.

- What are the unary operations in Relational Algebra?

PROJECTION and SELECTION are the unary operations in relational algebra.

- How do you communicate with an RDBMS?

Using Structured Query Language (SQL) RDBMS can be communicated.

- How many types of database

languages are? There are four

types of database languages:

1. Data Definition Language (DDL) e.g. CREATE, ALTER,

DROP etc. 2. Data Manipulation Language (DML) e.g.

SELECT, UPDATE, INSERT etc. 3. Data Control Language

(DCL) e.g. GRANT and REVOKE.

4. Transaction Control Language (TCL) e.g. COMMIT and ROLLBACK.

- What do you understand by Data Model?

Data model is specified as a collection of conceptual tools for describing data, data relationships, data semantics and constraints.

- Define a Relation Schema and a Relation.

A Relation Schema is denoted by $R(A_1, A_2, \dots, A_n)$ is made up of the relation name R and the list of attributes A_i that it contains.

A relation is specified as a set of tuples. See this example:

Let r be the relation which contains set tuples $(t_1, t_2, t_3, \dots, t_n)$. Each tuple is an ordered list of n values $t = (v_1, v_2, \dots, v_n)$.

- What is degree of a Relation?

The degree of relation is a number of attribute of its relation schema.

- What is Relationship?

Relationship is defined as an association among two or more entities.

- What are the disadvantages of file processing systems?
- Inconsistent
- Not secure
- Data redundancy
- Difficult in accessing data
- Data isolation
- Data integrity
- Concurrent access is not possible

- What is data abstraction in DBMS?

Data abstraction in DBMS is a process of hiding irrelevant details from users. Because Database systems are made of complex data structures so, it makes easy the user interaction with database.

- What are the three levels of data abstraction? Following are three levels of data abstraction:
- Physical level: It is the lowest level of abstraction. It describes how data are stored.
- Logical level: It is the next higher level of abstraction. It describes what data are stored in

database and what relationship among those data.

- View level: It is the highest level of data abstraction. It describes only part of entire database.

- What is DDL (Data Definition Language)?

Data Definition Language (DDL) is a standard for commands which define the different structures in a database. Most commonly DDL statements are CREATE, ALTER, and DROP.

- What is DML (Data Manipulation Language)?

Data Manipulation Language (DML) is a language that enable user to access or manipulate data as organised by appropriate data model.

There are two type of DML:

- Procedural DML or Low level DML: It requires a user to specify what data are needed and how to get those data.
- Non-Procedural DML or High level DML: It requires a user to specify what data are needed without specifying how to get those data.

- Explain the functionality of DML Compiler.

The DML Compiler translates DML statements in a query language that the query evaluation engine can understand.

- What is Relational Algebra?

Relational Algebra is a Procedural Query Language which contains a set of operations that take one or two relations as input and produce a new relation.

- What is Relational Calculus?

Relational Calculus is a Non-procedural Query Language which uses mathematical predicate calculus instead of algebra.

- What do you understand by query optimization?

The term query optimization specifies an efficient execution plan for evaluating a query that has the least estimated cost.

- What do you mean by durability in DBMS?

Once the DBMS informs the user that a transaction has successfully completed, its effects should persist even if the system crashes before all its changes are reflected on disk. This property is called durability.

- What is normalization?

Normalization is a process of analyzing the given relation schemas according to their functional dependencies. It is used to minimize redundancy and also minimize insertion,

deletion and update distractions.

- What is Denormalization?

Denormalization is the process of boosting up database performance and adding of redundant data which helps to get rid of complex data.

- What is functional Dependency?

Functional Dependency is the starting point of normalization. It exists when a relation between two attributes allows you to uniquely determine the corresponding attribute's value.

- What is E-R model?

E-R model is a short name for Entity Relationship model. This model is based on real world. It contains basic objects (known as entities) and relationship among these objects.

- What is entity?

Entity is a set of attributes in a database.

- What is an Entity type?

An entity type is specified as a collection of entities, having same attributes.

- What is an Entity set?

The entity set specifies the collection of all entities of particular entity type in the database.

- What is an Extension of entity type?

An extension of entity type is specified as a collection of entities of a particular entity type are grouped together into an entity set.

- What is Weak Entity set?

When an entity set doesn't have sufficient attributes to form a primary key, and its primary key comprises of its partial key and primary key of its parent entity, then it is called Weak Entity set.

- What is an attribute?

An attribute is a particular property, which describes the entity.

- What are the integrity rules in DBMS? There are

two integrity rules in DBMS:

- Entity Integrity: It specifies that "Primary key cannot have NULL value."
- Referential Integrity: It specifies that "Foreign Key can be either a NULL value or should be Primary Key value of other relation."

- What is Data Independence?

Data independence specifies that "the application is independent of the storage structure and access strategy of data".

It makes you able to modify the schema definition in one level should not affect the schema definition in the next higher level.

There are two types of Data Independence:

- Physical Data Independence: Modification in physical level should not affect the logical level.
- Logical Data Independence: Modification in logical level should affect the view level. NOTE: Logical Data Independence is more difficult to achieve.

- What are the three levels of data abstraction?

- Physical level: It is the lowest level of abstraction. It describes how data are stored. Logical level: It is the next higher level of abstraction. It describes what data are stored in database and what relationship among those data.
- View level: It is the highest level of abstraction. It describes only part of entire database.

- What is stored procedure?

A stored procedure is a named group of SQL statements that have been previously created and stored in the server database.

- What is 1NF?

1NF is the First Normal Form. It is the simplest type of normalization that you can implement in a database. The main objectives of 1NF are to:

- Remove duplicate columns from the same table
- Create separate tables for each group of related data and identify each row with a unique column

- What is 2NF?

2NF is the Second Normal Form. A table is said to be 2NF, if it follows the following conditions:

The table is in 1NF.

Every non-prime attribute is fully functionally dependent on primary key.

- What is 3NF?

3NF stands for Third Normal Form. A database is called in 3NF if satisfies the following conditions:

It is in second normal form.

There is no transitive functional dependency.

- What is BCNF?

BCNF stands for Boyce-Codd Normal Form. It is an advance version of 3NF so it is also

referred as 3.5NF. BCNF is stricter than 3NF.

A table complies with BCNF if it satisfies the following conditions: It is in 3NF. For every functional dependency $X \rightarrow Y$, X should be the super key of the table.

- What is PL SQL ?

PL SQL is a procedural language which has interactive SQL, as well as procedural programming language constructs like conditional branching and iteration.

- List the uses of database trigger.

A PL/SQL program unit associated with a particular database table is called a database trigger. It is used for :

- Audit data modifications.
- Log events transparently.
- Enforce complex business rules.
- Maintain replica tables
- Derive column values
- Implement Complex security authorizations

- What are the two types of exceptions.

Error handling part of PL/SQL block is called Exception. They have two types : user_defined and predefined.

- What are functions and procedures in a PL SQL block?

Function is called as a part of an expression. Procedure is called as a statement in PL/SQL.

- Explain Commit, Rollback and Savepoint. For a COMMIT statement, the following is true:

- Other users can see the data changes made by the transaction.
- The locks acquired by the transaction are released.
- The work done by the transaction becomes permanent.

A ROLLBACK statement gets issued when the transaction ends, and the following is true. The work done in a transaction is undone as if it was never issued.

All locks acquired by transaction are released. It undoes all the work done by the user in a transaction.

With SAVEPOINT, only part of transaction can be undone.

- How many triggers can be applied to a table?

A maximum of 12 triggers can be applied to one table.

- List 3 basic parts of a trigger.
- A triggering statement or event.
- A restriction

- An action

- Mention what PL/SQL package consists of? A PL/SQL package consists of

- PL/SQL table and record TYPE statements
- Procedures and Functions
- Cursors

- Variables (tables, scalars, records, etc.) and constants
- Exception names and pragmas for relating an error number with an exception
- Cursors

- Define Atomicity and Aggregation.

Atomicity: It's an all or none concept which enables the user to be assured of incomplete transactions to be taken care of. The actions involving incomplete transactions are left undone in DBMS.

Aggregation: The collected entities and their relationship are aggregated in this model. It is mainly used in expressing relationships within relationships.

- Enlist the various transaction phases. The various transaction phases are:

- Analysis Phase.
- Redo Phase
- Undo Phase

- What is Storage Manager?

It is a program module that provides the interface between the low-level data stored in database, application programs and queries submitted to the system.

- What is Buffer Manager?

It is a program module, which is responsible for fetching data from disk storage into main memory and deciding what data to be cache in memory.

- What is Transaction Manager?

It is a program module, which ensures that database, remains in a consistent state despite system failures and concurrent transaction execution proceeds without conflicting.

- What is File Manager?

It is a program module, which manages the allocation of space on disk storage and data structure used to represent information stored on a disk.

- What is Authorization and Integrity manager?

It is the program module, which tests for the satisfaction of integrity constraint and checks the authority of user to access data.

- Describe ODBC

ODBC is a standard that contains an interface that provides a common language for application programs to access and process SQL databases. In order to use ODBC, a driver, server name, database name, user id, and password are required. ODBC is important for Internet applications and has gained wide acceptance.

59). Provide an overview of XML.

XML is used to structure and manipulate data involved with a browser and is becoming the standard for e-commerce. XML uses tags that are similar to HTML in that they use the angle brackets, but XML describes the content whereas HTML describes the appearance. The XML schema standard was published in May 2001.

60). Describe Website security issues.

Website security issues include unauthorized access to the several aspects of one's Website.. Security measures should include all aspects of the system such as the network, operating level, database, and Web server. Regular monitoring and security testing by a company should help to avoid intrusion into one's system.

- Explain the role of metadata for the three-layer architecture.

Each of the three layers has a metadata layer linked with it. The metadata layer describes the properties or characteristics of the data. The operational metadata describe the data used in the various operational and external systems. The enterprise data warehouse metadata describe the reconciled data layer. The data mart metadata describes the derived data layer.

- Explain concurrency transparency.

Concurrency transparency is where each transaction in a distributed database is treated as if it is the only one in the system. Therefore if several transactions are running at one time. the results will be the same as if each transaction was run in serial order. The transaction manager helps to provide concurrency control.

The three methods that may be used are locking, versioning, and time stamping
.